EPFL - Computer Science

CS-401 Applied Data Analysis

Lecture 12 — 03/12/2025

Author: Jonathan Dutoit

Professor: Clément Pit-Clauded / Robert West

How to scale massive data

Warm up

```
count = 0
for n in range(0, 10**9):
   if n.bit_count() == 5:
        count += 1
print(count)
```

Question: how long is this going to take? 1 billion operations, suppose `n.bit_count()` is instantaneous (fast) and most CPU have optimization to that operation. Since modern CPUS run at 4-5GHz, one would expect it to take < 1 seconds. In the professor machine, it took surprisingly **51 seconds**.

Also, this program is not doing anything interesting. If we had an interesting computation there, it would be even slower. Like cross-referencing large datasets.

Another question: Does this course use Moodle? The Moodle admins have a responsibility to log connections to track how the website is being used. Suppose it takes a couple of Gigabytes, this would already **be more than 1 billion...**

One year of Moodle logs:

```
14000 students
*  3-20 session per day
*    4-7 days per week
*     14 week per term
*      2 terms per year
*     3-5 pages per session
*    10-80 requets per page (!)
*  200-500 bytes per log entry
= 5500GB = 5.5TB
=     10 hours to read from a hard drive??
```

$\Rightarrow$ distribute, parallelize, stream

So far in this class...

- We made one big assumption

  - All data fits on a single machine
  - Even more, all data fits into memory on a single machine (Pandas)
- Realistic assumption for **prototyping**, but frequently not for production code

⇒ You need to be able to program the cloud

The big-data problem

Data is growing fast than computation speed

+ Growing data sources

  (e.g., Web, mobile, sensors, ...)

+ Cheap hard-disk storage

- Slow read speeds ⇒ Must distribute

- Stalling CPU speeds ⇒ Must parallelize

- RAM bottlenecks ⇒ Must stream

Examples

Typical hard drive: $12 / TB (2024), 150 MB/s reads ⇒ 2h / TB

- All StackOverflow posts: 100 GB
- WSL (Swiss) rockslide measurements: 2 TB / day
- Worldwide stock-market data: 2.5 TB / day
- Github Arctic Code Vault: 21 TB
- Facebook daily logs, 2016: 60 TB / day
- Entire BrainMaps project: 140 TB
- Uber's analytics: PBs / day
- CERN's filtered experimental data: 1 PB / day
- Entire Wayback Machine archive: 99 PB
- CERN raw experimental data: 1 PB / second

But how much data should you get?

Of course, "it depends", but for many applications the answer is: **As much as you can get**

Big data about people (text, Web, social media) tends to follow heavy-tailed distributors (e.g., power laws). Example: Web search.

Note: **59% of all Web search queries are unique**. 17% of all queries are made only twice.

The big-data problem

A single machine can no longer store, let alone process, all the data. The solution is to **distribute** over a large cluster of machines.

Hardware for big data

- **Budget** (a.k.a. commodity) hardware. Not "gold-plated" (a.k.a. custom)
- Many low-end servers: **easy to add capacity**. **Cheaper** per CPU and per disk
- **Increased complexity in software:**
  - Fault tolerance
  - Virtualization (e.g., distributed file systems)

Problems with cheap hardware

Failures, e.g., (Google numbers)

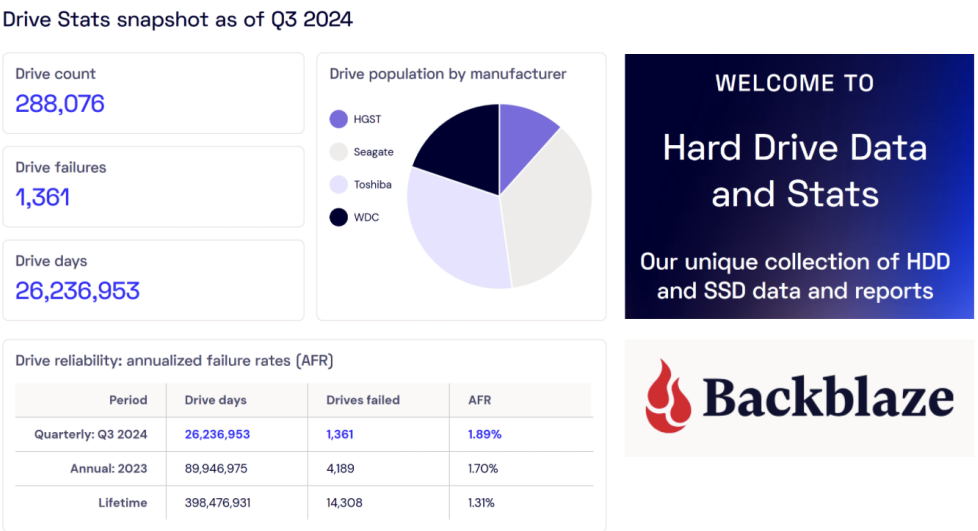
- 1-5% hard drives/year
- 0.2% DIMMs (dual in-line memory modules)/year

Commodity network (1-10 Gb/s) speeds vs. RAM

- Much more latency (100-100,000x)
- Lower throughput (100-1,000x)

Uneven performance

- Inconsistent hardware (e.g., old + new)
- Variable network latency
- External loads



Period	Drive days	Drives failed	AFR
Quarterly: Q3 2024	26,236,953	1,361	1.89%
Annual: 2023	89,946,975	4,189	1.70%
Lifetime	398,476,931	14,308	1.31%

What's hard about cluster computing?

1. **How to split work across machines?**
2. **How to deal with failures?**

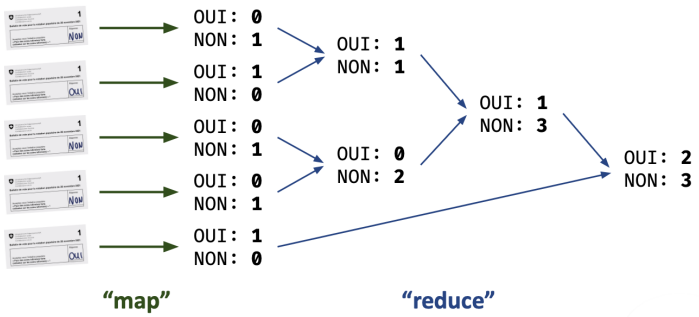
Distributed aggregation

Models of distributed computation



- First idea: **Truck full of ballots** going to Bern. Centralized collection, centralized counting. This is not ideal: shipping costs to transport all ballots. Also, voting participation is high in Switzerland: sequential count takes time!
- Second ideal: **Train full of more ballots** going to Bern. **Distributed** collection, centralized counting. Transportation costs were highly reduced.
- Third idea: **One phone call** to Bern. Centralized collection, **distributed counting**.
- Fourth idea: **partial counts and use phone calls**! **Distributed** collection, **distributed** counting.

All distributed:



MapReduce / Hadoop

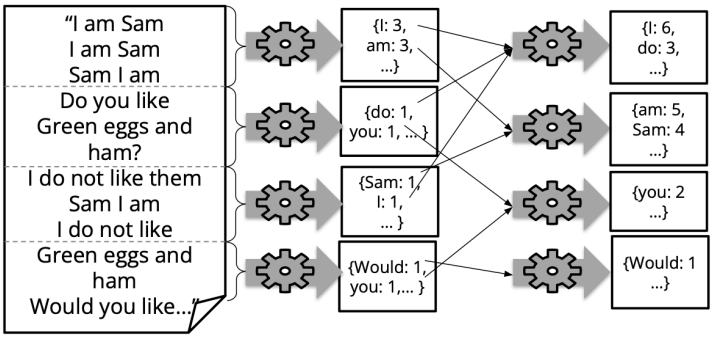
Models of distributed computation

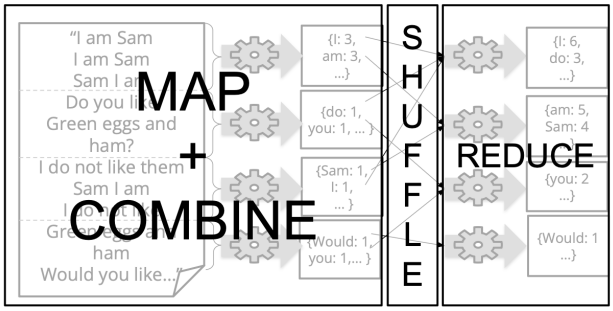
How do you count the number of occurrences of each word in a document? We could do a hashtable!

"I am Sam
[I] am Sam
Sam I am
Do you like
Green eggs and
ham?"

{I: 2, am: 1, Sam: 1}... What if the document is REALLY big?

Divide and conquer!





1. How to divide work across machines?
   - Moving data may be very expensive
   - Must consider network, data locality
2. How to deal with failures?
   - 1 server fails every 3 years $\Rightarrow$ 10K servers see $\sim$10 faults/day
   - Even worse: stragglers (node not failed, but slow)

Note: checking whether a computer is going to finish its computation it's a well-known undecidable problem: **Halting problem**!

Solution: MapReduce

- Smart systems engineers have done all the work for you (**hadoop**).
  - Task scheduling
  - Virtualization of file system
  - Fault tolerance (incl. data replication)
  - Job monitoring
  - etc.
- "All" you need to do: implement Mapper and Reducer classes
  Need to break more complex jobs into sequence of MapReduce jobs

How to deal with failures? Just launch another task!

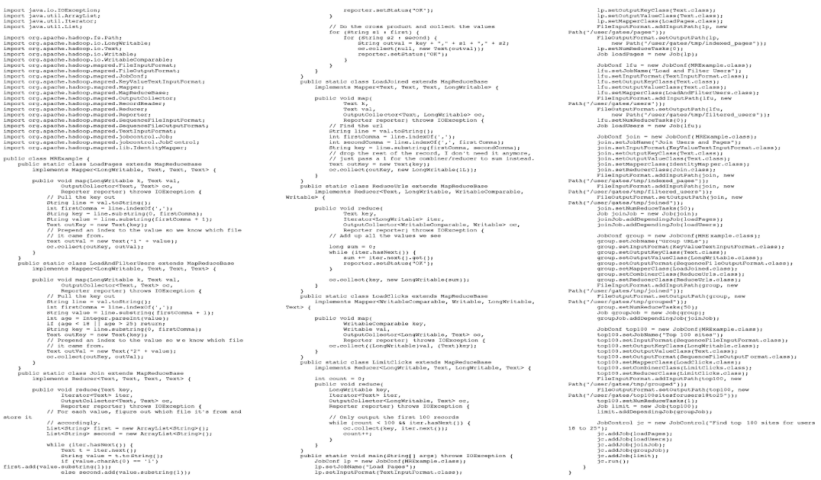
How to deal with slow tasks? Just launch another task!

Example Task

Suppose you have user info in one file, website logs in another, and you need to find the top 5 pages most visited by users aged 18–25



Example Task in MapReduce



Enter: Spark

- A high-level fun(ctional) programming API

```
print("I am a regular Python program, using pyspark")

users = (sc.textFile('users.tsv') # user <TAB> age
          .map(lambda line: line.split('\t'))
          .filter(lambda user_age: 18 <= int(user_age[1]) <= 25))

pages = (sc.textFile('pageviews/*.tsv')  # user <TAB> url
           .map(lambda line: line.split('\t')))

counts = (users.join(pages)
               .map(lambda user_age_url: (user_age_url[1][1], 1))
               .reduceByKey(lambda x, y: x + y)
               .takeOrdered(5))
```

This pretty closely mirrors the diagram and will faster than `mapReduce`. Actually, for distributed cloud computing style programs, keeping programs as high-level as possible gives them higher optimization opportunities.

Spark

- Implemented in **Scala** (go EPFL!)
- Additional APIs in
  - Python, Java, R
- Many abstractions
  - Core: RRDs
  - Derived: dataframes, SQL, ML toolkit

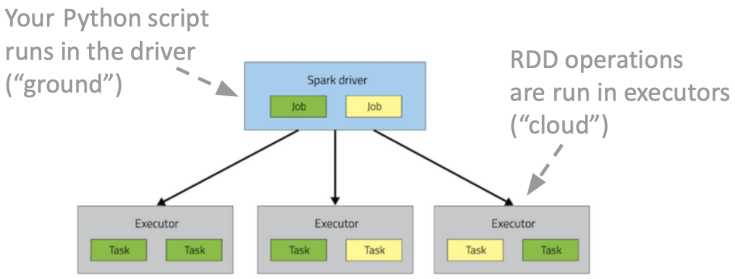
RDDs: concepts

Models of distributed computation

RDD: resilient distributed dataset

- To programmer: looks like one single collection (each elements represents a "row" of a dataset)
- Under the hood: oh boy...
  - RDDs "live in the cloud": split over several machines, replicated, etc.
  - Can be processed in parallel
  - Can be transformed to a single, real collection (if small...)
  - Typically read from a distributed file system (HDFS)
  - Can be written to a distributed file system

Spark architecture



- One executor is essentially one machine with multiple CPUs
- One task is the fact of processing one partition of data
- Each task is run by one CPU. So one executor can execute multiple tasks at the same time.

RDD operations

- **"Transformations"**
  - Input: RDD; output: another RDD
  - Everything remains "in the cloud"
  - Example: for every entry in the input RDD, count chars
    - RDD: {'I', 'am', 'you'} ⇒ RDD:{1, 2, 3}
- **"Actions"**
  - Input: RDD; output: a value that is returned to the driver
  - Result is transferred "from cloud to ground"
  - Example: take a sample of entries from RDD and print it on the driver's shell; or store results to file (local or distributed)

Lazy execution

- **Transformations** (i.e., RDD ⇒ RDD operations) are not executed until it's really necessary (a.k.a. "lazy execution")
- Execution of transformations triggered by **actions**
- Why?
  - If you never look at the data, there's no point in manipulating it...

Exercises

Rewrite the following to be more efficient:

- `rdd.sort().first() == rdd.min()`
- `rdd.map(f).map(g) == rdd.map(lambda x: g(f(x))`

Implement map and filter using only `flatMap`

- `rdd.map(f) == rdd.flatMap(lambda x: [f(x)])`
- `rdd.filter(f) == rdd.flatMap(lambda x: [x] if f(x) else [])`

RDD transformations

- **sample**`(withReplacement?, fraction, seed)`: Sample a *fraction* of the data, with or without replacement, using a given random number generator *seed*
- **union**`(otherDataset)`: New dataset that contains the union of the source and the argument.
- **intersection**`(otherDataset)`:...
- **distinct**`()`: New dataset that contains distinct elements of the source
- **groupByKey**(): When called on a dataset of (K, V) pairs, returns a dataset of `(K, Iterable<V>)` pairs.
- **reduceByKey**`(func)`: when called on dataset of (K, V) pairs, returns a dataset of (K, V) pairs where the values for each key are aggregated using the given reduce function `func`, which must be of type (V, V) => V
- **reduce**`(func)`: ?? (Needs associativity (unordered dataset) and commutativity (ordered dataset))
- **sortByKey**`()`: When called on a dataset of (K, V) pairs, returns a dataset of (K, V) pairs sorted by keys
- **join**`(otherDataset)|`: When called on datasets of type (K, V) pairs and (K, W) pairs, returns a dataset of (K, (V, W)) pairs with all pairs of elements for each key (default is a **left-join**)
- Analogous: **leftOuterJoin**, **rightOuterJoin**, **fullOuterJoin**

RDD actions

- **collect()**: Return all the elements of the dataset as an array at the driver program. This is usually useful after a filter or other operation that returns a sufficiently small subset of the data.
- **count**(): Return the number of elements in the dataset
- **reduce**`(func)`: Compute a single value (**deterministic?**)
- **take**`(n)`: Return an array with the "first"n elements of the dataset.
- **saveAsTextFile**`(path)`: Save to local filesystem or HDFS

Broadcast variables

```
my_set = set(...)
rdd2 = rdd1.filter(lambda x: x in
    my_set)
```

This is a bad idea: `my_set` needs to be shipped with every task (one task per data partition, so if `rdd1` is spread over N partitions, the above will require copying the same object N times)

Note: broadcast variables are **read-only**

```
my_set = sc.broadcast(set(...))
rdd2 = rdd1.filter(lambda x: x in
    my_set.value)
```

This way, `my_set` is copied to each executor only once and persists across all tasks (one per partition) on the same executor

Accumulators

```
def f(x): return x*2
rdd2 = rdd1.map(f)
```

How can we easily know how many rows there are in rdd1 (without running a costly reduce operation)?

Side effects via accumulators!

```
counter = sc.accumulator(0)
def f(x): counter.add(1); return x*2
rdd2 = rdd1.map(f)
```

- Accumulators are **write-only** ("add-only") for executors
- Only driver can read the value: counter.value

How does Spark run?

```
print("I am a regular Python program, using pyspark")

users = (sc.textFile('users.tsv') # user <TAB> age
           .map(lambda line: line.split('\t'))
           .filter(lambda user_age: 18 <= int(user_age[1]) <= 25))

views = (sc.textFile('pageviews/*.tsv')  # user <TAB> url
           .map(lambda line: line.split('\t')))

counts = (users.join(views)
               .map(lambda user_age_url: (user_age_url[1][1], 1))
               .reduceByKey(lambda x, y: x + y)
               .takeOrdered(5))
```

Dataframes and SQL

Models of distributed computation

Spark DataFrames

- Bridging the gap between your experience with Pandas and the need for distributed computing
  - RDD = collection of records
  - DataFrame = table with rows and typed columns

- Important to understand what RDDs are and what they offer but today most of the tasks can be accomplished with DataFrames

Spark DataFrames

```
users = spark.read.csv("users.csv", header=True)
views = spark.read.csv("pageviews/*.csv", header=True)
counts = (
    users[users.age.between(18, 25)]
      .drop("age")
      .join(views, on="name")
      .groupby("url").count()
      .sort_values("name", ascending=True)
      .head(5)
).toPandas()
```

Spark SQL

```
counts = spark.sql("""
  SELECT url, COUNT(*)
  FROM users
  JOIN views ON users.name == views.name
  WHERE 18 <= age AND age <= 25
  GROUP BY url
""")

counts.toPandas()
```